

INTRODUCTION TO SYSTEMS ANALYSIS AND DESIGN: AN AGILE, ITERATIVE APPROACH

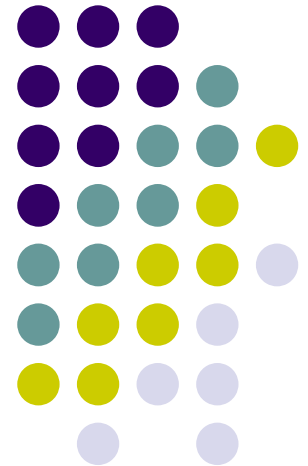
SATZINGER | JACKSON | BURD

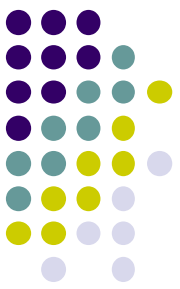
CHAPTER 12

Designing the User and System Interfaces

Chapter 12

Introduction to Systems
Analysis and Design:
An Agile, Iterative Approach
6th Ed
Satzinger, Jackson & Burd





Outline

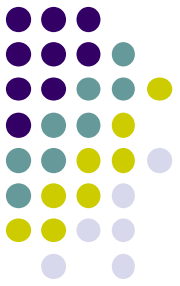
- Implementation and Development (实施和开发)
- Quality assurance (质量保障)
- Installation (安装)
- Documentation (文档)
- Training and user support (培训和用户支持)



第 12 章 面向对象实现

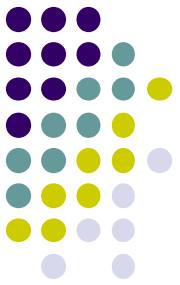
面向对象实现主要包括两项工作：

- 把面向对象设计结果翻译成用某种程序语言书写的面向对象程序；
- 测试并调试面向对象的程序。



12.1 启发规则

1. 设计结果应该清晰易懂
2. 一般 - 特殊结构的深度应适当
3. 设计简单的类
4. 使用简单的协议
5. 使用简单的服务
6. 把设计变动减至最小



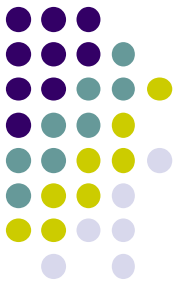
12.1 启发规则

1. 设计结果应该清晰易懂

- 提高软件可维护性和可重用性的重要措施
- 保证设计结果清晰易懂的主要因素：
 - (1) 用词一致
 - (2) 使用已有的协议
 - (3) 减少消息模式的数目
 - (4) 避免模糊的定义

2. 一般 - 特殊结构的深度应适当

- 类等级中包含的层次数适当
- 一个中等规模 (大约包含 100 个类) 的系统中, 类等级层次数应保持为 7 ± 2 。



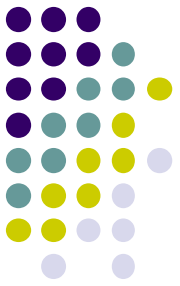
12.1 启发规则

3. 设计简单的类

- 尽量设计小而简单的类
- 注意以下几点：
 - (1) 避免包含过多的属性
 - (2) 有明确的定义
 - (3) 尽量简化对象之间的合作关系
 - (4) 不要提供太多服务

4. 使用简单的协议

- 消息中的参数不要超过 3 个



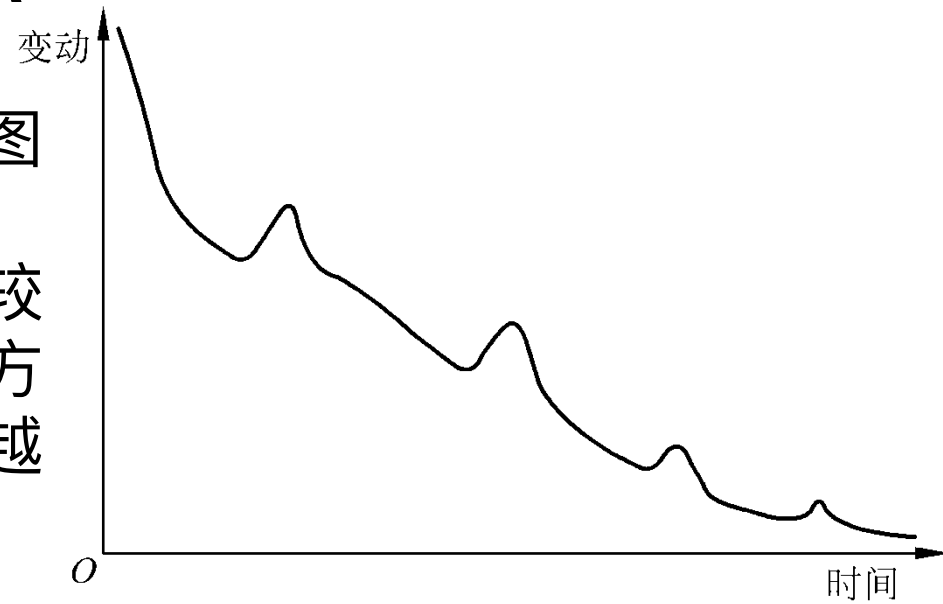
12.1 启发规则

5. 使用简单的服务

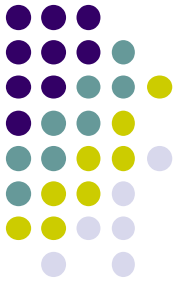
- 类中的服务通常都很小
- 尽量避免使用复杂的服务

6. 把设计变动减至最小

- 理想的设计变动曲线如右图所示
- 在设计的前期阶段，变动较大，随着时间推移，设计方案日趋成熟，改动也越来越小了



12.2 软件重用



12.2.1 概述

12.2.2 类构建

12.2.3 设计简单的类

12.2 软件重用

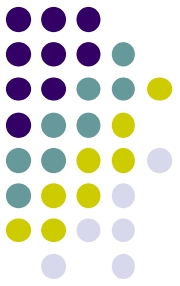


12.2.1 软件重用概述

1. 重用

- 重用也叫再用或复用，是指同一事物不作修改或稍加改动就多次重复使用。
- 广义地说，软件重用可分为以下 3 个层次：
 - (1) 知识重用
 - (2) 方法和标准的重用
 - (3) 软件成分的重用
- 上述前两个重用层次属于知识工程研究的范畴，本节仅讨论软件成分重用问题。

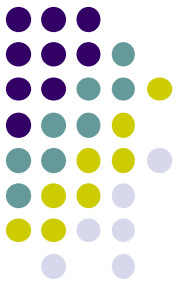
12.2 软件重用



2. 软件成分的重用级别

- 代码重用
 - (1) 源代码剪贴
 - (2) 源代码包含
 - (3) 继承
- 设计结果重用
重用某个软件系统的设计模型 (即求解域模型)
- 分许结果重用
更高级别的重用, 即重用某个系统的分析模型

12.2 软件重用

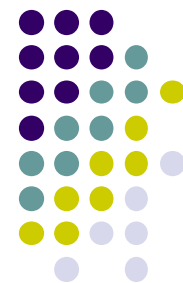


3. 典型的可重用软件成分

主要有以下 10 种：

- (1) 项目计划
- (2) 成本估计
- (3) 体系结构
- (4) 需求模型和规格说明
- (5) 设计
- (6) 源代码
- (7) 用户文档和技术文档
- (8) 用户界面
- (9) 数据
- (10) 测试用例

12.2 软件重用



12.2.2 类构件

面向对象技术中的“类”，是比较理想的可重用软构件，不妨称之为类构件。

1. 可重用软构件应具备的特点

- 为使软构件也像硬件集成电路那样，能在构造各种各样的软件系统时方便地重复使用，就必须使它们满足下列要求：
 - (1) 模块独立性强
 - (2) 成具有高度可塑性
 - (3) 接口清晰、简明、可靠
 - (4) 需求模型和规格说明

12.2 软件重用

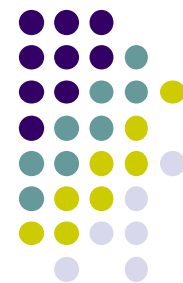


2. 类构件的重用方式

- 实例重用
 - (1) 使用适当的构造函数，按照需要创建类的实例
 - (2) 用几个简单的对象作为类的成员创建一个更复杂的类
- 继承重用

继承性提供了一种对已有的类构件进行裁剪的机制
- 多态重用
 - (1) 使对象的对外接口更加一般化，降低了消息连接的复杂程度
 - (2) 提供一种简便可靠的软构件组合机制

12.2 软件重用



11.3.3 软件重用的效益

1. 质量

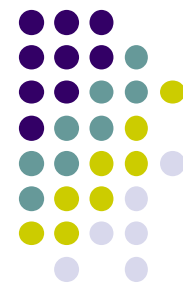
理想情况下，为了重用而开发的软件构件已被证明是正确的，且没有缺陷。

事实上，由于不能定期进行形式化验证，错误可能而且也确实存在。

但是，随着每一次重用，都会有一些错误被发现并被清除，构件的质量也会随之改善。

随着时间的推移，构件将变成实质上无错误的。

12.2 软件重用



2. 生产率

当把可重用的软件成分应用于软件开发的全过程时，创建计划、模型、文档、代码和数据所需花费的时间将减少，从而将用较少的投入给客户id提供相同级别的产品，因此，生产率得到了提高。

由于应用领域、问题复杂程度、项目组的结构和大小、项目期限、可应用的技术等许多因素都对项目组的生产率有影响，因此，不同开发组织对软件重用带来生产率提高的数字的报告并不相同，但基本上 30%~50% 的重用大约可以导致生产率提高 25%~40% 。



12.3 程序设计语言

开发人员在选择面向对象语言时，还应该着重考虑以下一些实际因素。

1. 将来能否占主导地位

- ❑ 为了使自己的产品在若干年后仍然具有很强的生命力，人们可能希望采用将来占主导地位的语言编程。
- ❑ 根据目前占有的市场份额，以及专业书刊和学术会议上所做的分析、评价，人们往往能够对未来哪种面向对象语言将占据主导地位做出预测。
- ❑ 但是，最终决定选用哪种面向对象语言的实际因素，往往是诸如成本之类的经济因素而不是技术因素。



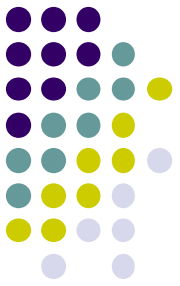
12.3 程序设计语言

2. 可重用性 语言、开发环境和类库这 3 个因素综合起来，共同决定了可重用性。

采用面向对象方法开发软件的基本目的和主要优点，是通过重用提高软件生产率。因此，应该优先选用能够最完整、最准确地表达问题域语义的面向对象语言。

3. 类库和开发环境

- 考虑类库的时候，不仅应该考虑是否提供了类库，还应该考虑类库中提供了哪些有价值的类。
- 为便于积累可重用的类和重用已有的类，在开发环境中，除了提供前述的基本软件工具外，还应该提供使用方便的类库编辑工具和浏览工具。其中的类库浏览工具应该具有强大的联想功能。

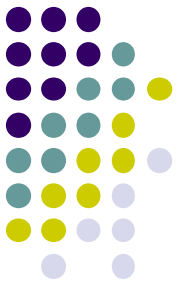


12.3 程序设计语言

4. 其他因素

在选择编程语言时，应该考虑的其他因素还有：

- ❑ 对用户学习面向对象分析、设计和编码技术所能提供的培训服务；
- ❑ 在使用这个面向对象语言期间能提供的技术支持；
- ❑ 能提供给开发人员使用的开发工具、开发平台、发行平台；
- ❑ 对机器性能和内存的需求；
- ❑ 集成已有软件的容易程度等。



12.4 程序设计风格

12.4.1 提高可重用性

1. 提高方法的内聚

一个方法 (即服务) 应该只完成单个功能。如果某个方法涉及两个或多个不相关的功能, 则应该把它分解成几个更小的方法。

2. 可重用性

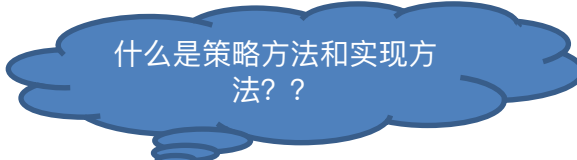
应该减小方法的规模, 如果某个方法规模过大 (代码长度超过一页纸可能就太大了), 则应该把它分解成几个更小的方法。



12.4 程序设计风格

3. 保持方法的一致性

保持方法的一致性，有助于实现代码重用。一般说来，功能相似的方法应该有一致的名字、参数特征（包括参数个数、类型和次序）、返回值类型、使用条件及出错条件等。



什么是策略方法和实现方法？

4. 把策略与实现分开

为提高可重用性，在编程时不要把策略和实现放在同一个方法中，应该把算法的核心部分放在一个单独的具体实现方法中。为此需要从**策略方法**中提取出具体参数，作为调用**实现方法**的变元。



12.4 程序设计风格

从所完成的功能看，有两种不同类型的方法

- 一类方法负责做出决策，提供变元，并且管理全局资源，可称为**策略方法**。

策略方法应该检查系统运行状态，并处理出错情况，它们并不直接完成计算或实现复杂的算法。策略方法通常紧密依赖于具体应用，这类方法比较容易编写，也比较容易理解。

- 另一类方法负责完成具体的操作，但却并不做出是否执行这个操作的决定，也不知道为什么执行这个操作，可称为**实现方法**。

实现方法仅仅针对具体数据完成特定处理，通常用于实现复杂的算法。实现方法并不制定决策，也不管理全局资源，如果在执行过程中发现错误，它们应该只返回执行状态而不对错误采取行动。由于实现方法是自含式算法，相对独立于具体应用，因此，在其他应用系统中也可能重用它们。



12.4 程序设计风格

5. 全面覆盖

如果输入条件的各种组合都可能出现，则应该针对所有组合写出方法，而不能仅仅针对当前用到的组合情况写方法。此外，一个方法不应该只能处理正常值，对空值、极限值及界外值等异常情况也应该能够作出有意义的响应。

6. 尽量不使用全局信息

应该尽量降低方法与外界的耦合程度，不使用全局信息是降低耦合度的一项主要措施。



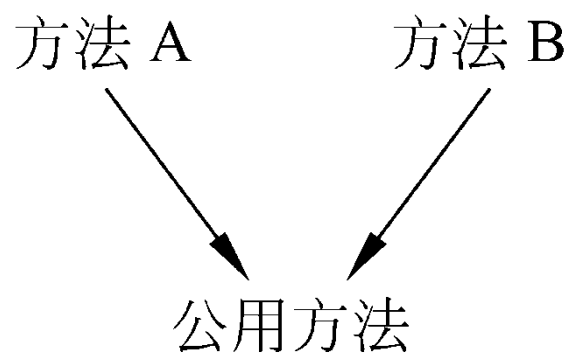
12.4 程序设计风格

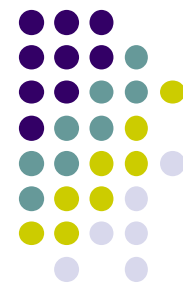
7. 利用继承机制

在面向对象程序中，使用继承机制是实现共享和提高重用程度的主要途径。

(1) 调用子过程

最简单的做法是把公共的代码分离出来，构成一个被其他方法调用的公用方法。如下图所示：

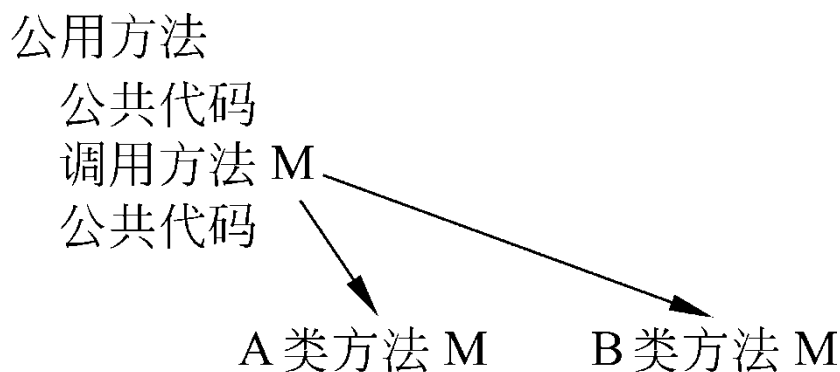


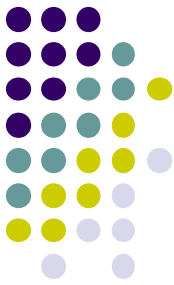


12.4 程序设计风格

(2) 分解因子

有时提高相似类代码可重用性的一个有效途径，是从不同类的相似方法中分解出不同的“因子”（即不同的代码），把余下的代码作为公用方法中的公共代码，把分解出的因子作为名字相同算法不同的方法，放在不同类中定义，并被这个公用方法调用，如图所示。使用这种途径通常额外定义一个抽象基类，并在这个抽象基类中定义公用方法。





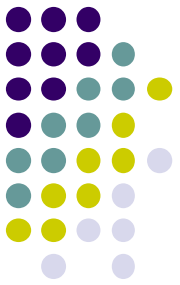
12.4 程序设计风格

(3) 使用委托

- 当确实存在一般 - 特殊关系时，使用继承才是恰当的。继承机制使用不当将造成程序难于理解、修改和扩充。
- 当逻辑上不存在一般 - 特殊关系时，为重用已有的代码，可以利用委托机制。

(4) 把代码封装在类中

程序员往往希望重用用其他方法编写的、解决同一类应用问题的程序代码。重用这类代码的一个比较安全的途径，是把被重用的代码封装在类中。



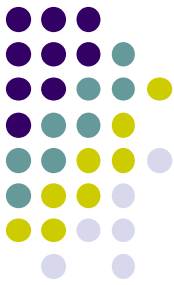
12.4 程序设计风格

12.4.2 提高可扩充性

上一小节所述的提高可重用性的准则，也能提高程序的可扩充性。此外，下列的面向对象程序设计准则也有助于提高可扩充性。

1. 封装实现策略

应该把类的实现策略（包括描述属性的数据结构、修改属性的算法等）封装起来，对外只提供公有的接口，否则将降低今后修改数据结构或算法的自由度。



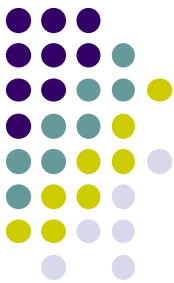
12.4 程序设计风格

2. 不要用一个方法遍历多条关联链

一个方法应该只包含对象模型中的有限内容。违反这条准则将导致方法过分复杂，既不易理解，也不易修改扩充。

3. 避免使用多分支语句

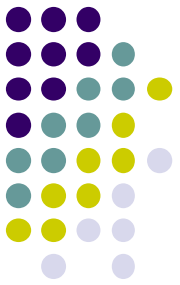
一般说来，可以利用 DO_CASE 语句测试对象的内部状态，而不要用来根据对象类型选择应有的行为，否则在增添新类时将不得不修改原有的代码。应该合理地利用多态性机制，根据对象当前类型，自动决定应有的行为。



12.4 程序设计风格

4. 精心确定公有方法

公有方法是向公众公布的接口。对这类方法的修改往往会涉及许多其他类，因此，修改公有方法的代价通常都比较高。为提高可修改性，降低维护成本，必须精心选择和定义公有方法。私有方法是仅在类内使用的方法，通常利用私有方法来实现公有方法。删除、增加或修改私有方法所涉及的面要窄得多，因此代价也比较低。



12.4 程序设计风格

12.4.3 提高健壮性

程序员在编写实现方法的代码时，既应该考虑效率，也应该考虑健壮性。

所谓**健壮性**就是在硬件故障、输入的数据无效或操作错误等意外环境下，系统能做出适当响应的程度。

必须认识到，对于任何一个实用软件来说，健壮性都是**不可忽略的质量指标**。



12.4 程序设计风格

1. 预防用户的操作错误

软件系统必须具有处理用户操作错误的能力。任何一个接收用户输入数据的方法，对其接收到的数据都必须进行检查，即使发现了非常严重的错误，也应该给出恰当的提示信息，并准备再次接收用户的输入，不能运行中断或“死机”。

2. 检查参数的合法性

对公有方法，尤其应该着重检查其参数的合法性，因为用户在使用公有方法时可能违反参数的约束条件。



12.4 程序设计风格

3. 不要预先确定限制条件

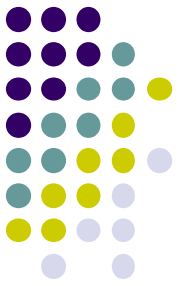
在设计阶段，往往很难准确地预测出应用系统中使用的数据结构的最大容量需求。因此不应该预先设定限制条件。如果有必要和可能，则应该使用动态内存分配机制，创建未预先设定限制条件的数据结构。

4. 先测试后优化

为在效率与健壮性之间做出合理的折衷，应该在为提高效率而进行优化之前，先测试程序的性能。经过测试，合理地确定为提高性能应该着重优化的关键部分。

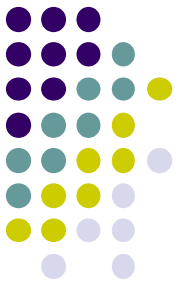
Order of Implementation

(continued)



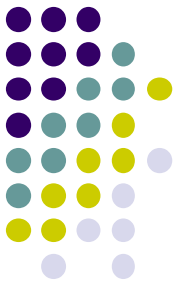
- Top-down and bottom-up order from traditional structured design and structured programming (传统结构化设计和结构化编程的自顶向下和自底向上顺序)
- **Top-down (自顶向下)** begins with module at top of structure chart
 - Always a working version of program
 - Requires three or more iterations to complete
- **Bottom-up (自底向上)** begins with modules at lowest level of structure chart
 - Many programmers can begin immediately
 - Requires driver programs to test

Team-Based Program Development



- Management issues （管理问题）
 - Organization of programming teams
 - Task assignment to specific teams or members
 - Member and team communication and coordination
- Variety of different models used for teams （各种不同的模型用于团队）
 - Cooperating peer, chief developer, collaborative specialist

Comparison and Summary of Development Team Types



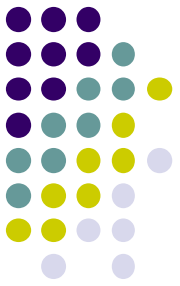
Team type	Team characteristics	Task and project types
Cooperating peers	Equal skill levels Overlapping specialties Consensus-based decision making	Experimentation Creative problem solving
Chief developer	Organized as a military platoon or squad One leader makes all important decisions	Well-defined objectives Well-defined path to completion
Collaborative specialists	Wide variation in skill and experience Minimal overlap in technical specialties Leader is primarily an administrator Consensus-based decision making	Diagnosis or experimentation Creative and integrative problem solving Wide range of technology

合作伙伴
解决

一样的技能，一样的专业，以共识为基础的决策

实验，创造性问题

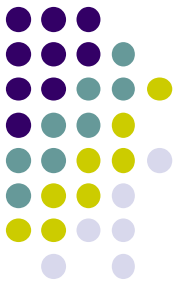
Comparison and Summary of Development Team Types



Team type	Team characteristics	Task and project types
Cooperating peers	Equal skill levels Overlapping specialities Consensus-based decision making	Experimentation Creative problem solving
Chief developer	Organized as a military platoon or squad One leader makes all important decisions	Well-defined objectives Well-defined path to completion
Collaborative specialists	Wide variation in skill and experience Minimal overlap in technical specialities Leader is primarily an administrator Consensus-based decision making	Diagnosis or experimentation Creative and integrative problem solving Wide range of technology

首席开发人员 像军队排或班一样组织，领导者做全部重要决策 明确的目标和实现方式

Comparison and Summary of Development Team Types

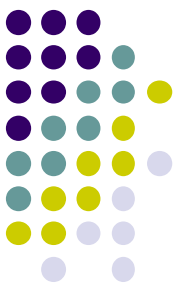


Team type	Team characteristics	Task and project types
Cooperating peers	Equal skill levels Overlapping specialties Consensus-based decision making	Experimentation Creative problem solving
Chief developer	Organized as a military platoon or squad One leader makes all important decisions	Well-defined objectives Well-defined path to completion
Collaborative specialists	Wide variation in skill and experience Minimal overlap in technical specialties Leader is primarily an administrator Consensus-based decision making	Diagnosis or experimentation Creative and integrative problem solving Wide range of technology

协作专家

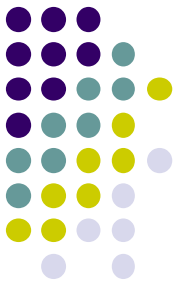
技能和经验的广泛变化，尽量减少技术专业的重叠
诊断或实验，创造性和综合性问题解决

Leader 主要是一个管理员，以共识为基础的决策
技术范围广



Source Code Control

- Source code control system (SCCS) 源代码控制系统
 - Automated tool for tracking source code files and controlling changes to those files (用于跟踪源代码文件和控制这些文件变更的自动化工具)
- Repository of code and programmer actions (代码和程序员操作的存储库)
 - Check out file in read-only mode (只读)
 - Check out file in read/write mode (读 / 写)
 - Check in a modified file (修改)

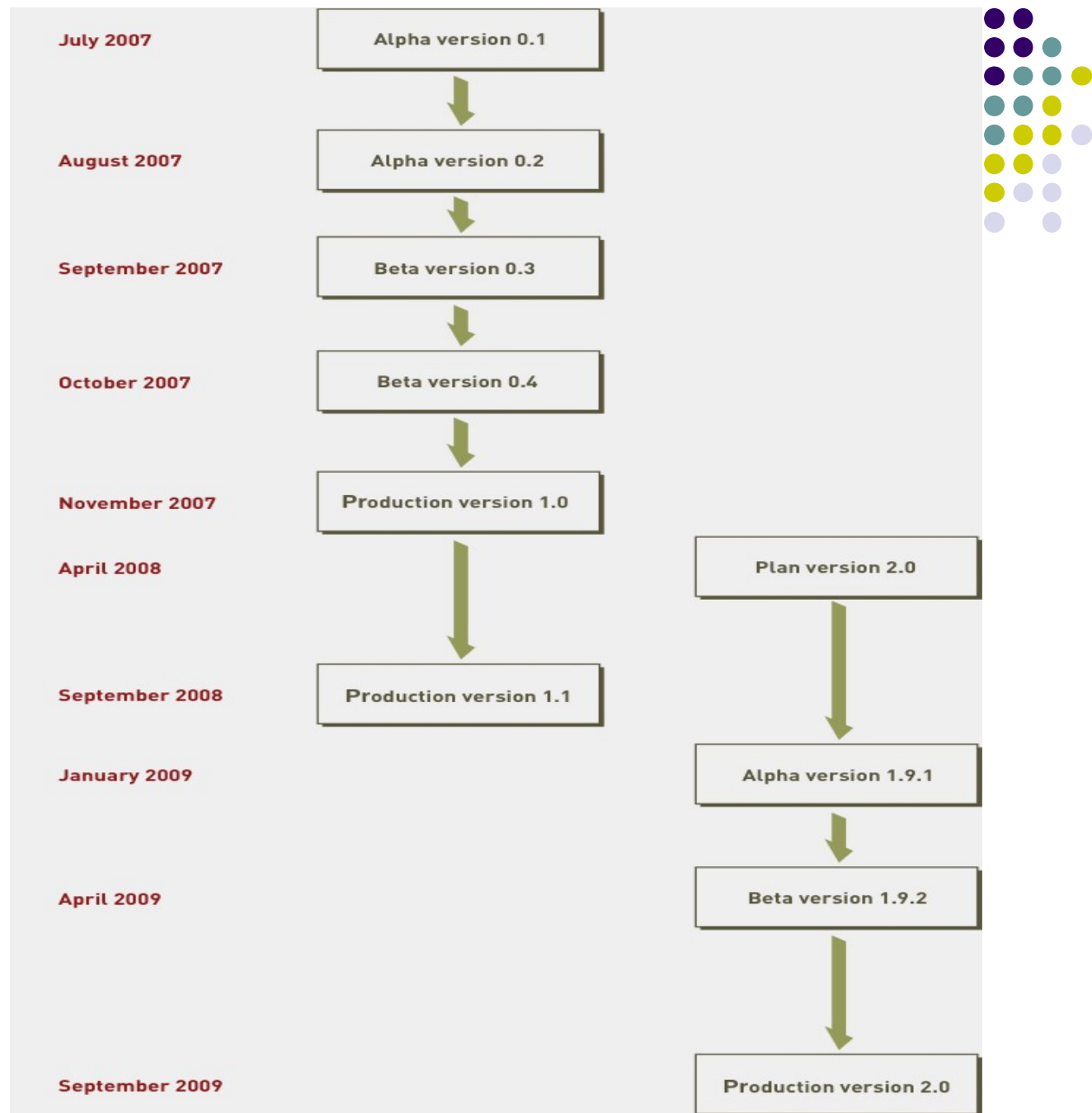


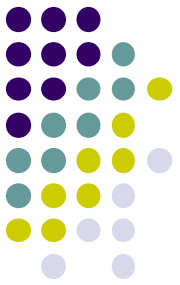
Versioning (版本管理)

- Mechanism to manage systems changes (管理系统变更的机制)
- Complex systems developed, installed, and maintained in series of versions to simplify testing and support (以一系列版本开发、安装和维护的复杂系统，以简化测试和支持)
 - **Alpha version** – incomplete testing version
 - **Beta version** – end-user testing version
 - **Production release version** – formally distributed to users or made operational
 - **Maintenance release** – bug fixes, small changes

(RMO 系统的测试和生产版本时间表)

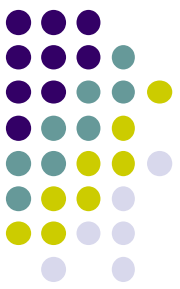
(RMO 系统的测试和生产版本时间表)





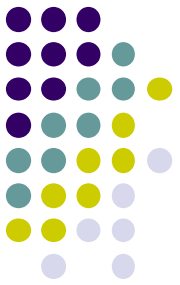
Outline

- Implementation and Development
- **Quality assurance (质量保证)**
- Installation
- Documentation
- Training and user support



Quality Assurance

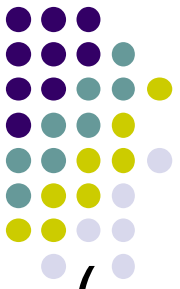
- Process of ensuring information system meets minimum quality standards （确保信息系统达到最低质量标准的过程）
- Determined by users, implementation staff, management （由用户、员工和管理人员来确定）
- Identification of gaps or inconsistencies in system requirements （识别系统需求中的差距或不一致）



Quality Assurance

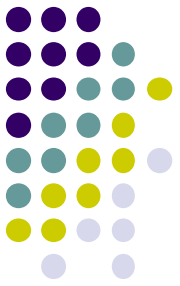
- QA integrated into project throughout SDLC
(将质量保证集成到项目的生命周期中)
- Cost of fixing errors rise as project progresses
(修复错误的成本随着项目的进展而上升)

Technical Reviews (技术评审)

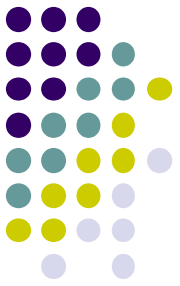


- Formal or informal reviews of design or construction details by group of developers (由开发者小组对设计或施工细节进行正式或非正式的评审)
- Open design and construction process to input from other people (开放设计和施工过程，听取其他人的意见)
- Other programmers can frequently see errors missed by original programmer (其他程序员可以看到原始程序员的错误)

Technical Reviews (技术评审)

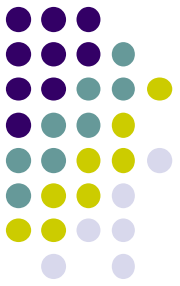


- Similar to author writing and editor reviewing (类似于作者写作和编辑评审)
- Walkthroughs (演练) and inspections (检查)
 - Reduce number of errors by factor of 5 to 10 (将错误数量减少 5-10 倍)
 - Reduce testing costs by 50% (降低 50% 的测试成本)



Testing (测试)

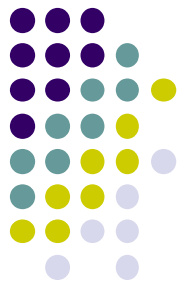
- Process of examining a product to determine if any defects exist (检查产品以确定是否存在缺陷的过程)
- Testing levels are related to specific SDLC phases (测试级别与特定的生命周期阶段相关)
- Testing activities spread throughout SDLC (测试活动遍布整个生命周期)
- Most of testing takes place following software construction and definition of defect standards (大多数测试都是在软件构建和缺陷标准定义之后进行的)



Unit Testing (单元测试)

- Tests individual modules (模块测试) of code or methods before integrating with other software
- **Driver module** (驱动模块) used for testing
 - Sets values of input parameters (设置输入参数的值)
 - Calls module to be tested and passes input parameters (调用要测试的模块并传递输入参数)
 - Accepts return parameters from tested module (接受被测试模块的返回参数)
- **Stub testing** (存根测试) – test module simulates module not yet developed (模拟尚未开发的模块)

Integration Testing （集成测试）



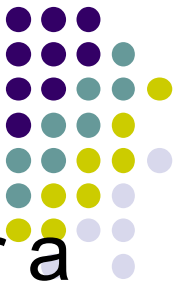
- Tests the behavior of a group of modules （一组模块） or methods
- Tests both normal processing and exceptions （测试正常过程和异常）
- Errors can include （错误包括）
 - Interface incompatibility （接口不兼容）
 - Incorrect parameter values （参数值不正确）
 - Run-time exceptions （运行时异常）
 - Unexpected state interactions （意外状态相互作用）

System Testing (系统测试)



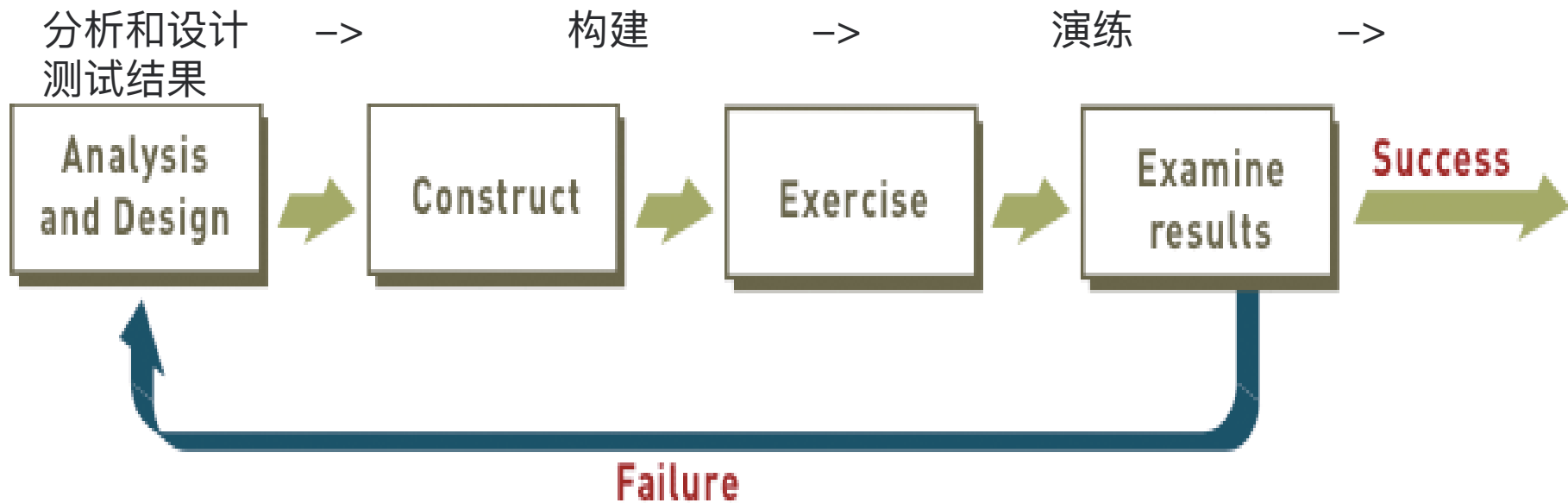
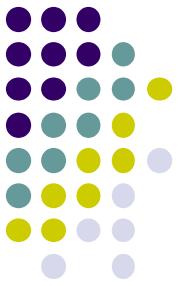
- Tests the behavior of the entire system (测试整个系统)
- **Build and smoke test** is performed daily to discover any problems with daily builds (构造和冒烟测试是日常执行或每周执行几次的系统测试)
 - System is completely compiled and linked each day (系统完全编译)
 - Battery of tests are run to smoke out problems (运行一系列测试以找出问题)
 - Any errors must be from changes made the prior day (任何错误肯定是前一天更改的)

Usability Testing (可用性测试)

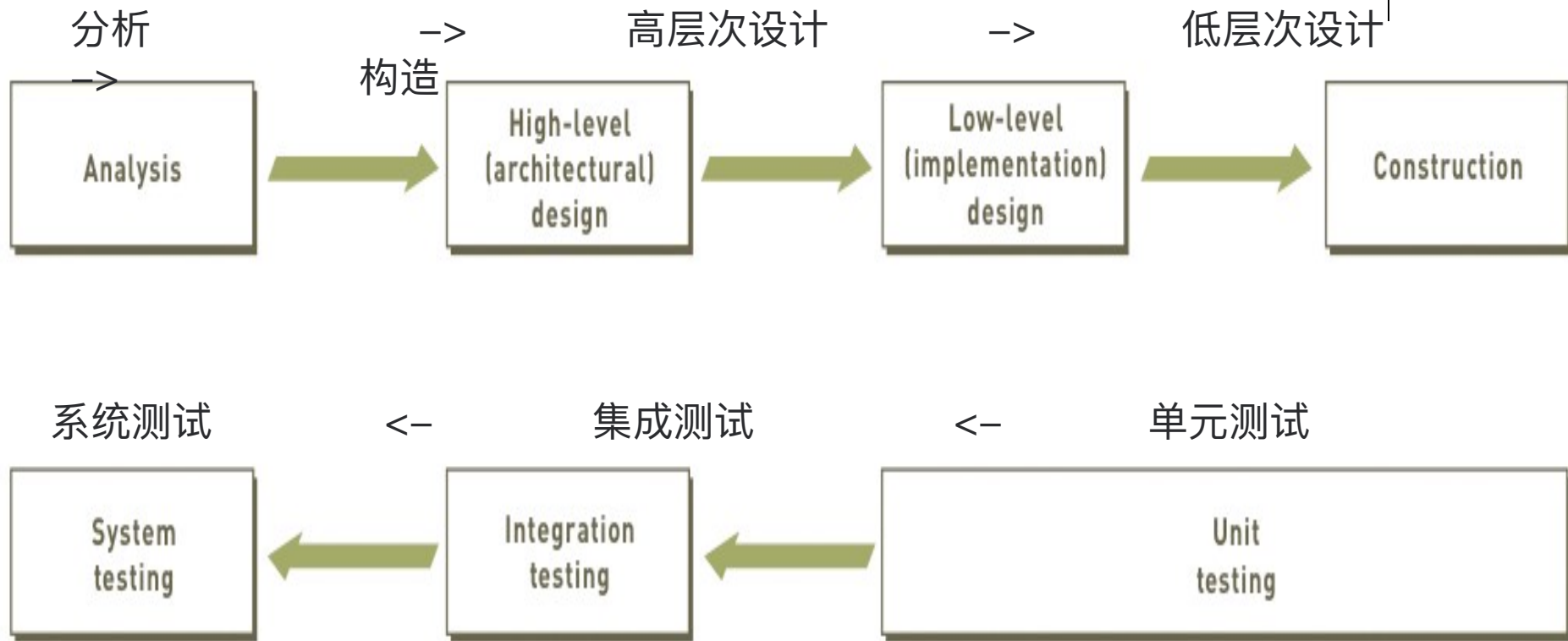
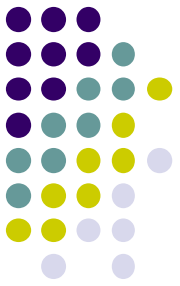


- **Usability test** is a test to determine whether a module, method, class, subsystem, or system meets user requirements (可用性测试是确定模块、方法、类、子系统或系统是否满足用户需求的测试)
- **Performance test** checks time-based requirements (性能测试检查基于时间的需求)
 - Response time (响应时间)
 - Throughput (吞吐量)
- **Acceptance test** is system test performed to determine whether system meets user requirements (验收测试确定系统是否满足用户需求)

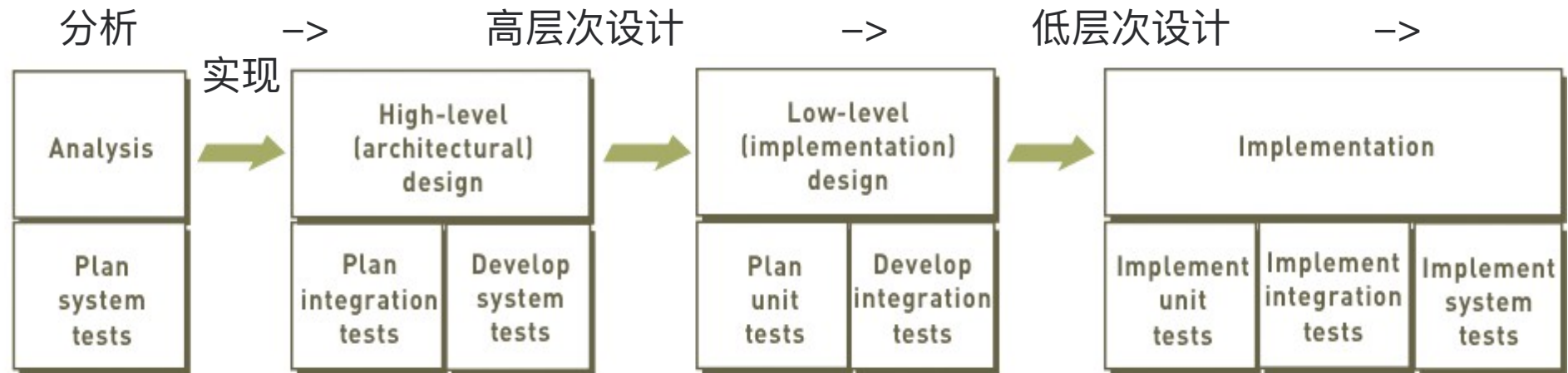
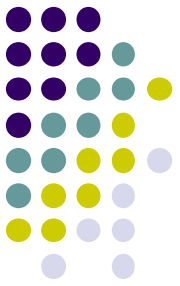
Generic Model of Software Testing

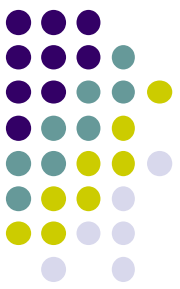


Correspondence Between SDLC Phases and Various Types of Testing



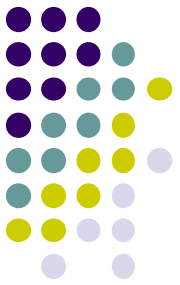
SDLC Phases and Testing Activities Performed Within Each Phase





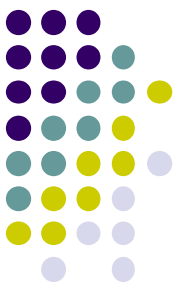
Test Cases (测试用例)

- Important part of testing is specifying test cases and test data (测试的重要部分是指定测试用例和测试数据)
- A **test case** is a formal description of (测试用例是一个正式的描述)
 - Starting state (开始状态)
 - Events to which software responds (软件响应的事件)
 - Expected response or ending state (预期的响应或结束状态)
- Analysis phase documentation is useful in preparing test cases (use-case driven)
- **Test data** is defined to be used with a test case



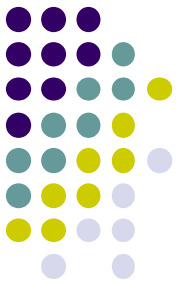
Test Cases (测试用例)

- Analysis phase documentation is useful in preparing test cases (use-case driven) (分析阶段文档在准备测试用例时是有用的)
- **Test data** is defined to be used with a test case (测试数据被定义为与测试用例一起使用)



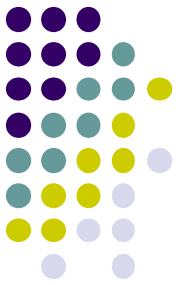
Data Conversion (数据转换)

- Data needed at system startup (系统启动时所需的数据)
 - Files or databases of system being replaced (被替换的系统文件或数据库)
 - Manual records (手工记录)
 - Files or databases of other systems (组织内其他系统的文件或数据库)
 - User feedback during normal system operation (系统正常运行时的用户反馈)



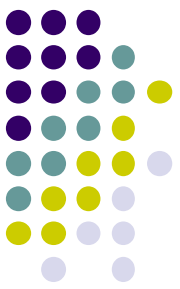
Data Conversion (数据转换)

- Reuse of existing databases (现有数据库的重要)
- Reloading database contents (重新加载数据库内容)
- Creating new databases (创建新数据库)



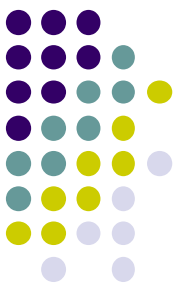
Outline

- Implementation and Development
- Quality assurance
- **Installation (安装)**
- Documentation
- Training and user support



Installation (安装)

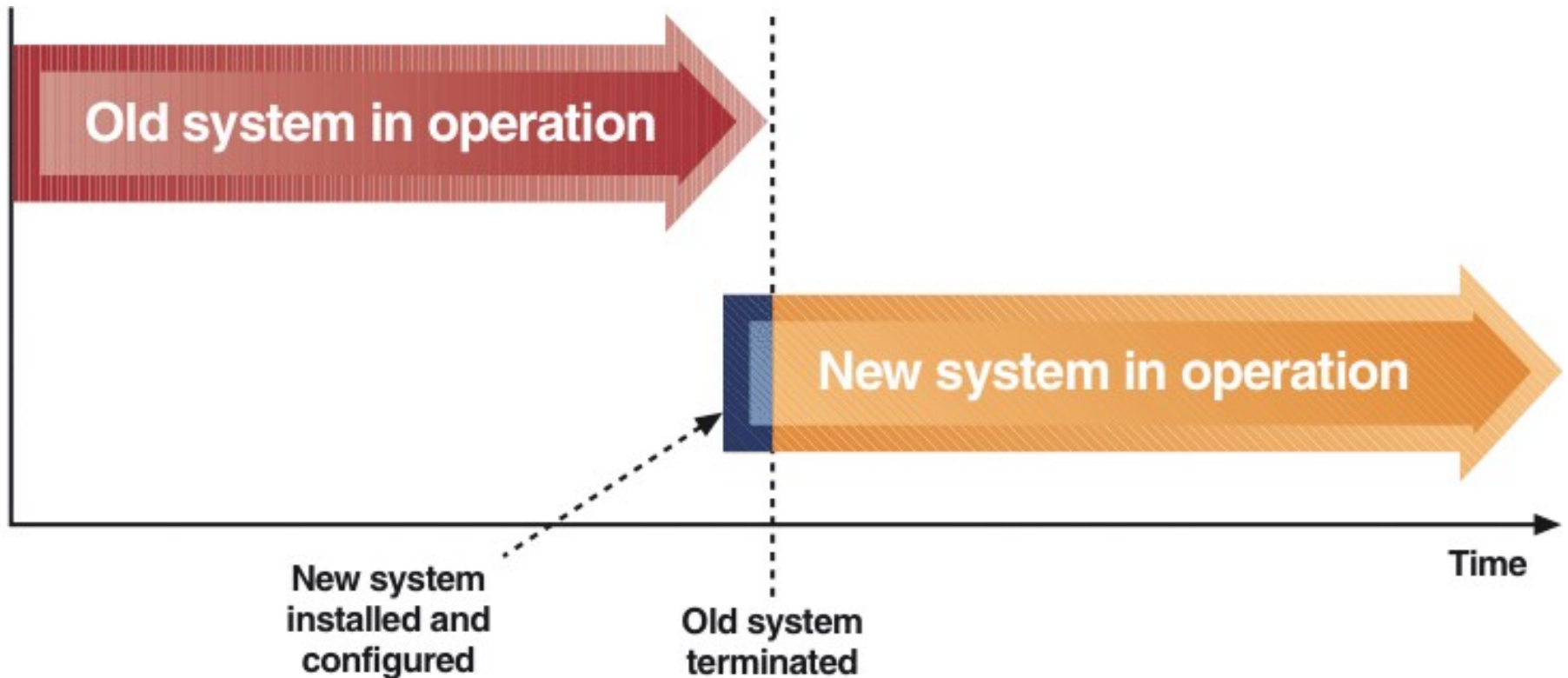
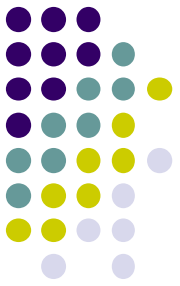
- After development and testing, system must be put into operation (开发和测试后, 系统需要投入运行)
- Important planning considerations (重要事项)
 - Costs of operating both systems in parallel (并行版本)
 - Detecting and correcting errors in new system (错误)
 - Potentially disrupting the company and IS operations (可能会扰乱公司信息系统正常运行的潜在因素)
 - Training personnel and customers with new procedures (培训员工, 使其熟悉新程序)

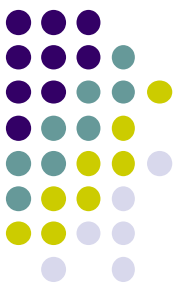


Direct Installation (直接部署)

- New system installed and quickly made operational (直接使用新系统)
- Overlapping systems turned off (关闭重叠系统)
- Advantage (优点) – simplicity and fewer logistics issues to manage (简单、缺少后勤问题)
- Disadvantage (缺点) – risk due to no backup (没有备份)

Direct Installation and Cutover

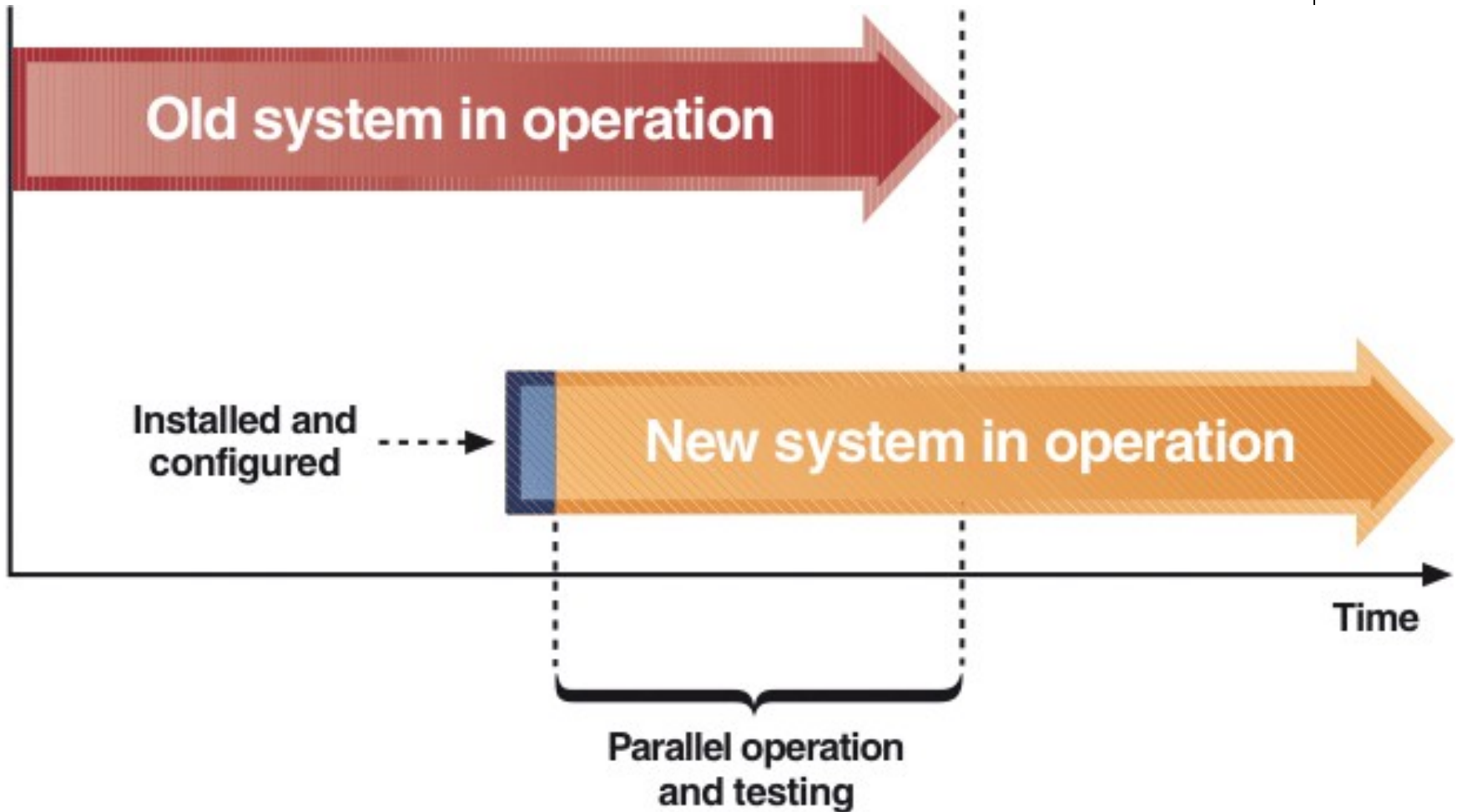
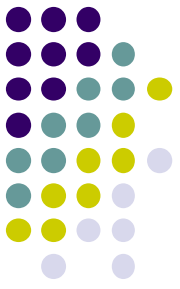




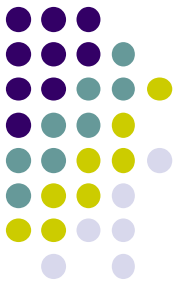
Parallel Installation (并行部署)

- Old and new systems operated together for extended period of time (旧系统和新系统一起运行很长时间)
- Advantages (优点) – low risk of system failure and continual backup (风险低、有备份)
- Disadvantage (缺点) – cost to operate both systems (操作两个系统的成本)
 - Hiring temporary personnel (人员)
 - Acquiring extra space (空间)
 - Increasing managerial and logistical complexity

Parallel Installation and Operation

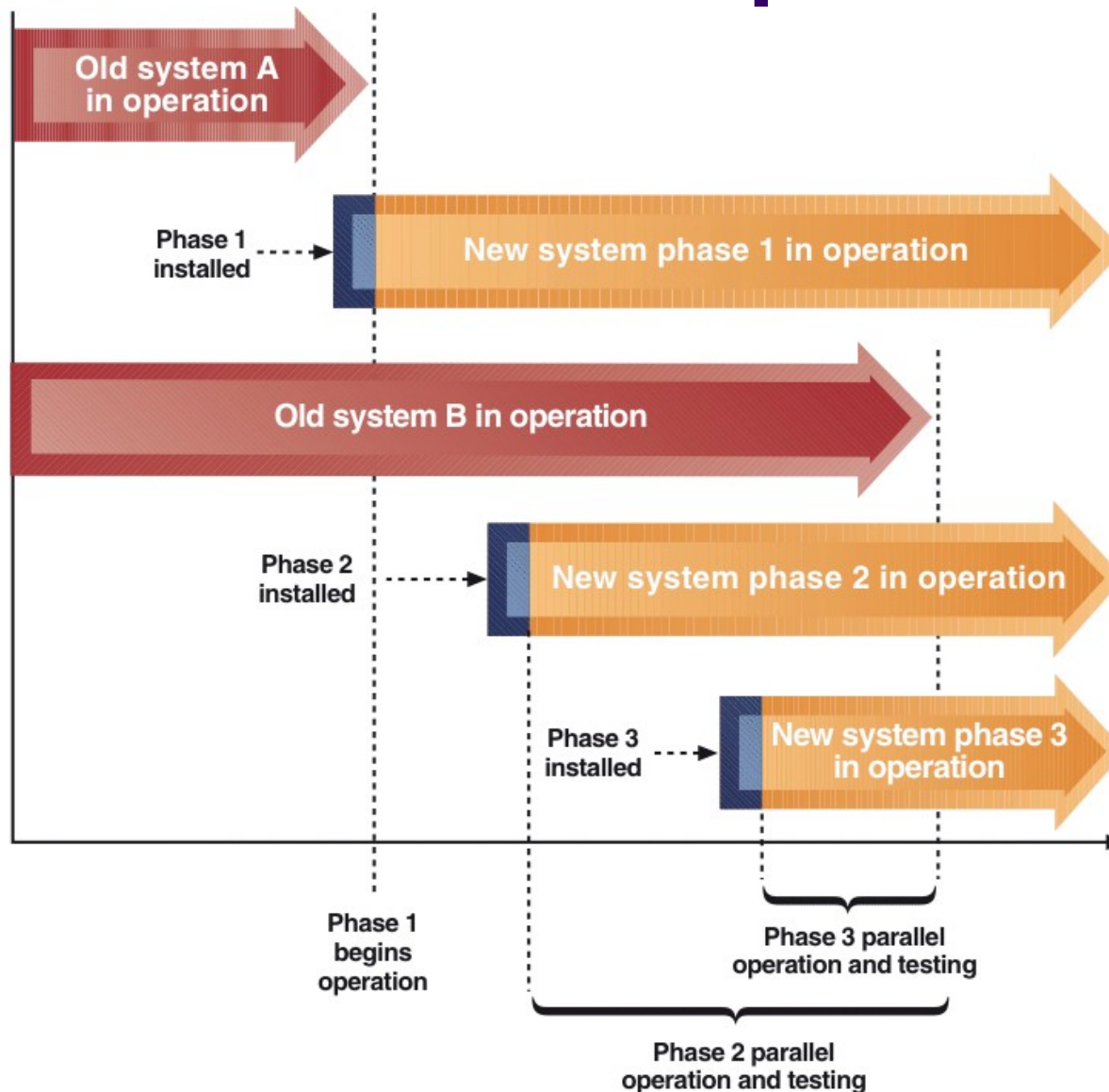
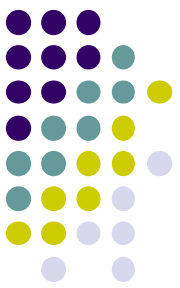


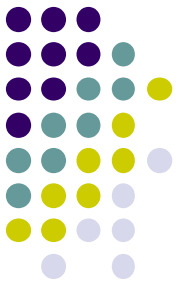
Phased Installation (阶段部署)



- New system installed in series of steps or phases (新系统按照步骤或阶段安装)
- Each phase adds components to existing system (每个阶段都向现有系统添加组件)
- Advantage (优点) – reduces risk because phase failure is less serious than system failure (降低风险)
- Disadvantage (缺点) – multiple phases cause more activities, milestones, and management complexity for entire effort (管理复杂)

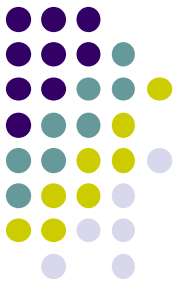
Phased Installation with Direct Cutover and Parallel Operation





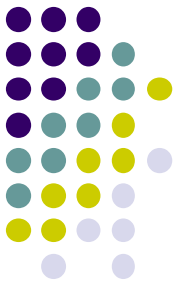
Outline

- Implementation and Development
- Quality assurance
- Installation
- Documentation (文档)
- Training and user support



Documentation

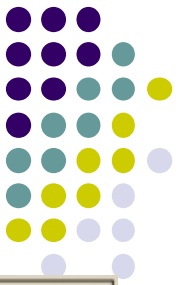
- Automated documentation is standard
 - Electronic manuals in MS Word or Adobe PDF format (电子手册)
 - Hyperlinked documents (超链接) – Web-browser formatted
 - Online documentation (在线文档) on vendor Web site
 - Embedded documentation on CD (嵌入式文档)
 - Electronic system model stored in graphic formats
 - Tool-specific system models developed with IDEs, DBMSs, and CASE tools



System Documentation

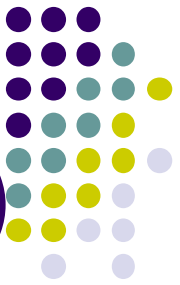
- Descriptions of system functions, architecture, and construction details （对系统功能、体系结构和构造细节的描述）
- Used by maintenance personnel and future developers（供维护人员和开发人员使用）
- Generated as a by-product of development
 - Includes source code
 - Includes analysis and design models
- Failure to maintain system documentation compromises value of a system

Life Cycle Phases and System Documentation Generated in Each Phase

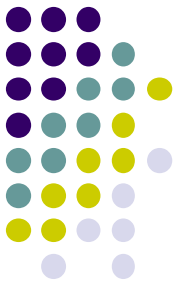


Life cycle phase	System documentation	
	Traditional approach	Object-oriented approach
Analysis	Entity-relationship diagram Data flow diagram Process description Data flow and element definition	Domain class diagram Use case Activity diagram System sequence diagram
	Event list	
Design	System flowchart Structure chart	Design class diagram Interaction diagram Communication diagram Package diagram State machine diagram
	Module or method pseudocode Database schema diagram	
Implementation	Program source code Database schema source code Test data	

User Documentation (用户文档)

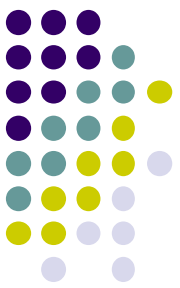


- Descriptions of how to interact with and maintain the system (如何与系统交互和维护系统描述)
- Used by end users (用户) and system operators (系统操作员)
- Topics include
 - Startup and shutdown (开始于关闭)
 - Keystrokes, mouse, or command functions to perform specific functions (按键、命令功能)
 - Program function for specific business procedures
 - Common errors and correction techniques (常见错误)



Outline

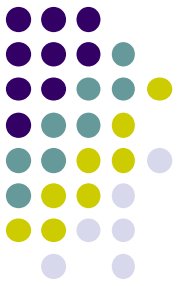
- Implementation and Development
- Quality assurance
- Installation
- Documentation
- Training and user support (培训和用户支持)



Training and User Support

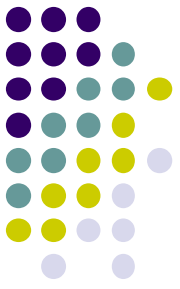
- Without training, user error rates will be high (如果没有培训，用户错误率将很高)
- Training considerations (培训注意事项)
 - Frequency and duration of system use (系统使用的频率和持续时间)
 - Need to understand system's business context (业务环境)
 - Existing computer skills and proficiency (现有的计算机技能和熟练程度)

Ongoing Training and User Support



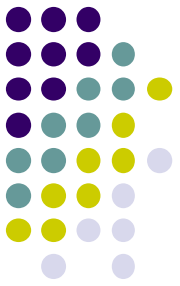
- User support covers training and user assistance that occurs after installation (用户支持包括安装后的培训和用户协助)
 - Online documentation and troubleshooting (在线文档和故障排除)
 - Resident experts (常驻专家)
 - Help desk (帮助台)
 - Technical support (技术支持)

Maintenance and System Enhancement

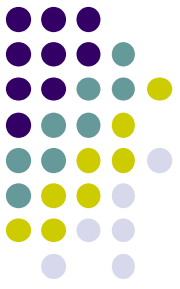


- Modification of software after delivery to correct faults, improve performance, or adapt the product to a changed environment (维护及系统提升)
 - Tracking modification requests and changes (跟踪修改请求和变更)
 - Implementing changes (实施更改)
 - Monitoring system performance (监控系统性能)
 - Upgrading hardware and software (升级硬件和软件)
 - Updating documentation (更新文档)

Submitting Change Requests and Error Reports



- Most organizations adopt formal change control procedures to manage change risks (大多数组织采用正式的变更控制过程来管理风险)
 - Standard change request forms
 - Review of requests by change control committee
 - Extensive planning for design and implementation
- Approved changes are added to list of pending changes for budgeting, scheduling, planning, and implementation
- A separate process is used for error correction



Implementing a Change

- Planning for a change includes (变更计划包括：)
 - Identifying parts of system to change or add
 - Securing personnel to implement change
 - Scheduling design and implementation activities
 - Developing test criteria and test plan for changed system
- System documentation is reviewed to **determine scope of change** (审核系统文档以确定变更的范围)



A Change Request Example

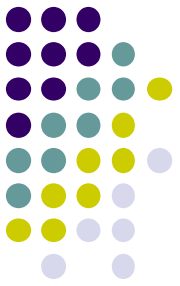
Change Request			
Request Date	2/1/2007	Change Type	☐ Error Correction
Requested By	Wen-Hsu Chang, Comptroller		☒ Modification
Target System	Customer Accounts - Refunds		☐ New Function
Change (or Error) Description			
U.S. check formats will soon change due to a recently enacted federal law. The new format reserves an area to the right of the current routing number to be used for a security bar code checksum.			
The law requires the new checksum to be printed on checks dated on or after January 1, 2008.			
We currently use a portion of the area in question to print a multicolored security symbol. The security symbol will need to be moved or eliminated, and the security bar code checksum will have to be added.			
美国账单的格式由于刚制定的联邦法律二变更了			

法律要求新的账单格式应当在 2008.1.1 更新

A Change Review Form

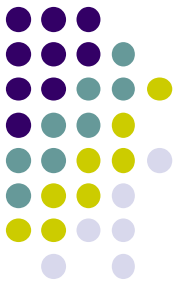


Change Review			
Change Request ID	2007-11	Date Reviewed	2/7/2007
Priority	☐ Critical ☒ Necessary ☐ Optional		
Hardware Implications need to verify ability of current printers to write a security bar code in mandated area			
Software Implications database will need to be modified to store the security bar code with other check information check writing program must be modified to generate and print the security bar code			
Performance Implications none			
Operating Budget Implications none			
Other Implications none			
Disposition	☒ Approved ☐ Rejected ☐ Suspended		
Reason			
Latest Implementation Date	12/31/2007		
Reevaluation Date	n/a	Signature	



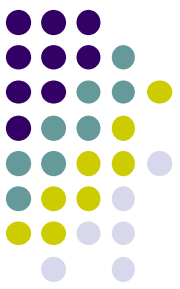
Summary

- Implementation activities occur after design and before system is turned over to users （实施活动发生在设计之后和系统交付用户之前）
- Implementation is complex （实现是复杂的）
 - Interdependence of programming, quality assurance, hardware and software installation, documentation, and training （编程、质量保证、硬件和软件安装、文档和培训的相互依赖）



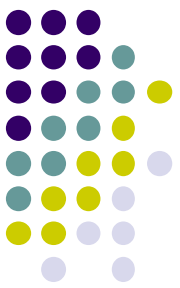
Summary (continued)

- Implementation is difficult to manage (实施很难管理)
 - Activities must be properly sequenced (活动必须有适当的顺序)
 - Progress must be continually monitored (不断监测进展情况)
- Implementation is risky (实施是有风险的)
 - Significant time and resources required (需要大量时间和资源)
 - Often affects systems vital to daily operations (经常影响对日常操作至关重要的系统)



Summary (continued)

- Software components constructed to (软件组件构造遵循：)
 - Minimize development resources needed (尽量减少所需的开发资源)
 - Maximize ability to test system and control errors (最大限度地测试系统和控制错误的能力)
 - These goals often conflict: trade-off among resources, time, and desire to correct errors (这些目标经常发生冲突：在资源、时间和纠正错误的愿望之间进行权衡)



Summary (continued)

- Data conversion, installation, documentation, and training follow programming and testing (数据转换、安装、文档和培训遵循编程和测试)
- Installed and documented system is prerequisite for complete training (安装并记录系统是完成培训的先决条件)
- Fully populated database needed to begin operation (开始操作需要完全填充的数据库)
- Support activities occur after system becomes operational and might continue for years to support user requirements and reduce operational risk (支持活动发生在系统可操作后)